L2lidar Class – Framework

May 24, 2026
V 1.3.2

Author: Mark Stegall

Platform: Qt6.10.2 (Qt Creator), C++
Hardware: Unitree L2 LiDAR (Ethernet / UDP)

L2lidar is the core class that implements a complete UDP communications pipeline, packet reconstruction, decoding, and synchronization logic without any UI dependencies. It is designed to be embedded into GUI or headless applications (Qt timers, ROS bridges, visualization tools, etc.).

It is intended to be used by applications that need to interface to the Unitree L2 4D LiDAR hardware via the UDP Ethernet interface.

Example Apps that use the L2lidar class:
L2diagnostic is a platform-independent C++ framework for validating and diagnosing the operation of the Unitree L2 LiDAR sensor over its Ethernet (UDP) interface. It relies on the L2lidarClass to control and receive data from the L2.
l2lidar_node is a platform-independent C++ framework for a ROS2 publishing node for IMU and point cloud data from the L2.

Goal

The project was developed to replace reliance on undocumented vendor libraries and POSIX-dependent archive binaries provided by Unitree Robotics, enabling transparent packet decoding, timestamp correction, latency measurement, and point-cloud extraction in a portable Qt environment.

Motivation

Unitree’s official SDK distributes:

- Header file
- Example applications
- Precompiled `.a` archive libraries

However:

The archive library source is unavailable:

- It relies on POSIX I/O
- Debugging and porting are difficult
- Protocol behavior is undocumented

This project reconstructs and documents the L2 protocol behavior while providing:

- Robust packet handling
- Error detection and packet loss statistics
- Timestamp synchronization to host time
- Quaternion-based spatial correction
- Extracted point cloud frames for downstream processing

Key Features

UDP Packet Reconstruction

* Combines multiple datagrams into complete L2 packets
* Detects lost and malformed packets

Protocol Decoding

- 3D point cloud packets
- 2D scan packets
- IMU packets
- Version, timestamp, MAC, IP, work mode, and parameter packets
- ACK packets

Point Cloud Conversion

- ConvertL2data2pointcloud() produces clean `Frame` (vector of `PCpoint`)
- Optional IMU-based quaternion spatial correction
- Includes precomputed range (meters) for each point
- Override of builtin range calibration
- Accurate timestamps relative to the host
- Accurate conversion of scan data point to cloud point

Time Synchronization

- Sync L2 clock to host system time
- Adjustable timestamp scale correction
- Host-driven periodic resynchronization

Latency Measurement

- RTT latency using sequence IDs
- EWMA statistics (average, variance, min, max)
- Non-blocking implementation

Command & Control

- Start/stop rotation
- Reset device
- Set work mode
- Configure UDP IP/port
- Set MAC address
- Retrieve firmware version and parameters

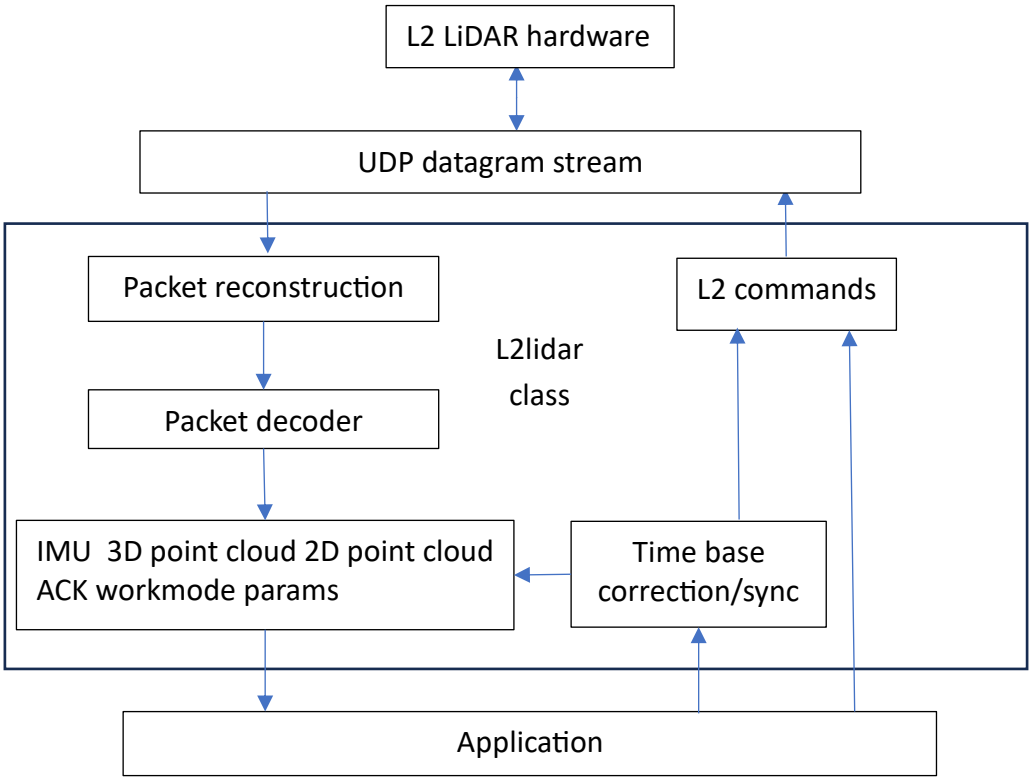
Thread-Safe Data Access

- All decoded packets protected by `QMutex`
- Safe access from GUI or worker threads

Qt-Native Design

- Uses `QUdpSocket`, `QTimer`, `QObject` signals
- No UI code included
- Ready for integration into Qt or ROS2 applications

Architecture Overview



Dependencies

- Qt (QObject, QUdpSocket, QTimer, QMutex, QByteArray, QVector)
- Qt6 Core shared library (Qt6Core) 6.10.2
- Qt6 Network shared library (Qt6Network) 6.10.2
- Modified Unitree protocol headers:
    - unitree_lidar_protocolL2.h
    - unitree_lidar_utilitiesL2.h
      original source: https://github.com/unitreerobotics/unilidar_sdk2
      BSD-3 Clause

Design Notes

- No UI elements are included in `L2lidar`
- UART support exists as commented reference code only. UART support has not been implemented
- Communication uses Ethernet (UDP)
- Timestamp correction compensates known firmware clock drift
- Initial IMU and point cloud frames are skipped until time synchronization stabilizes

L2lidar class

Only one instance of this class should be created in application. The Ethernet does not allow multiple connections unless multicasting is used. The L2 is a unicast device not capable of multicasting.

The dependencies are included in the l2lidar.h file as #includes.

The Unitree external dependencies that are needed by this class:

include “unitree_lidar_protocolsL2.h”
include “unitree_lidar_utilitiesL2.h”

These are modified versions of the original open-source files from the Unitree Lidar SDK2:

unitree_lidar_protocols.h (do not use this version)
unitree_lidar_utilities.h (do not use this version)

The other class dependencies are:

```
#include <QByteArray>
#include <QElapsedTimer>
#include <QMutex>
#include <QObject>
#include <QUdpSocket>
#include <QHostaddress>
#include <QTimer>
#include <QVector>
#include <unordered_map>
#include “quaternion.h”
#include “PCpoint.h”
using Frame = QVector<PCpoint>;
```

L2lidar Class files:

L2lidar.h
L2lidar.cpp

```
// This structure is used for the RTT network latency measurements
typedef struct {
   double lastMeasurement;
   double Average;
   double Variance;
   double min;
   double max;
} Latency;

// This structure is used for point cloud data
// It is contained in the file PCpoint.h
struct PCpoint
{
   float x;
   float y;
   float z;
   float intensity; //  0-255
   float range;    // meters
   long long time; // timestamp in nanoseconds
   uint32_t ring;
};

// This structure is used for point cloud data
// It is contained in the file quaternion.h
struct Quaternion
{
   float w, x, y, z;
};
```

Class L2lidar

public:

L2lidar(QObject* parent); // constructor
~L2lidar(); // uses default destructor

// Accessors to retrieve the latest L2 packets
const LidarAckData **ack**()
uint32_t **GetL2Workmode**()
const LidarImuDataPacket **imu**()
const LidarIpAddressConfig **IPaddress**() // This has not been observed
const LidarParamDataPacket **L2ParamsPacket**()
const LidarMacAddressConfig **MAC**() () // This has not been observed
const Lidar2DPointDataPacket **Pcl2Dpacket**()
const LidarPointDataPacket **Pcl3Dpacket**()
const LidarTimeStampData **timestamp**() // This has not been observed
const LidarVersionData **version**()

// UDP error reporting
const QString **GetLastUDPError**()

// packet stats from L2 (only updates when L2 socket connected)
// These are not actually critical, only for reporting stats
void **ClearCounts**(); // clears the packet totals
const Latency GetLatency()

// packets recieved from the L2
uint64_t **lostPackets**()
uint64_t **total2D**()
uint64_t **total3D**()
uint64_t **totalACK**()
uint64_t **totalIMU**()
uint64_t **totalPackets**()
uint64_t **totalOther**()

// packets retrieved by the app

```
uint64_t totalIMUretrieved()
uint64_t totalPCretrieved()

// L2 commands
bool GetL2Params(void);
bool GetWorkMode();
bool LidarGetVersion(void);
bool LidarReset(void);
bool LidarStartRotation(void);
bool LidarStopRotation(void);
bool sendLatencyID(uint32_t SeqeunceID);
bool SetL2MAC(LidarMacAddressConfig MACsettings); // requires reset
bool setL2UDPconfig(QString hostIP, uint32_t hostPort,
                    QString LidarIP, uint32_t LidarPort); // requires reset
bool SetWorkMode(uint32_t mode);

// L2 Timstamp correction and controls
void EnableL2TimeCorrection(bool enableflag;
void GetL2TimeScale(long long& ScaleNumerator, long long& ScaleDenominator);
void SetL2TimeScale(long long ScaleNumerator, long long ScaleDenominator);

// L2 timestamp syncing to host (timer driven)
void EnableL2TSsync(bool enable);
void SetL2TSsyncRate(uint32_t Rate);

// latency measurement
void EnableLatencyMeasure(bool enable);

// Time sync of L2 to host
bool SyncL2Clock() ;    // sync L2 to current system time
 bool SyncL2Clock(TimeStamp timestamp)
// this is only to set the UDP parameters in the class
// It DOES NOT change the L2 configuration settings
void LidarSetCmdConfig(
          QString srcIP, uint32_t srcPort,
          QString dstIP, uint32_t dstPort);
```

```
// UDP socket
bool ConnectL2();  // bind to create, bind socket, connect callback for decode
void DisconnectL2();   // close socket

// Conversion of point frame data from L2 to point cloud
   bool ConvertL2data2pointcloud(Frame& frame, bool Frame3D, bool IMUadjust,
                        bool CalOVR, double CalScale, double CalBias);

// L2 range calibration override
   void EnableCalibrationOVR(bool Override) {  // true: use override calibration
      OverideCalibration = Override;         // false: use internal calibration
   }

   void SetCalibrationOVR(double Scale, double Offset) {
      RangeScaleOVR = Scale;
      RangeBiasOVR = Offset;
   }

   bool GetCalibration(double& Scale, double& Offset){
```

signals:

```
   void ackReceived();
   void imuReceived();
   void IPreceived();
   void MACReceived();
   void L2ParamsReceived();
   void PCL2DReceived();
   void PCL3DReceived();
   void timestampReceived();
   void versionReceived();
   void WorkmodeReceived();
```

Signals are callbacks used to notify a subscriber that an event has occurred.

CLASS PUBLIC METHODS

Class constructor/destructor

L2lidar::L2lidar(QObject* parent);

~L2lidar::L2lidar();

L2 connect/disconnect methods

These are the equivalent to opening or closing the L2 communications.
To open communications with the L2 you must:

L2lidar::LidarSetCmdConfig()
L2lidar::ConnectL2()

To close you should:

L2lidar::DisconnectL2()

Note: The interface is automatically closed when the L2lidar class is deleted.

bool L2lidar::L2lidar::ConnectL2();

Currently only UDP Ethernet communications is supported.
The UART implementation is only a placeholder for future implementation.
You should set these settings before calling connectL2():

LidarSetCmdConfig() (if not called factory default settings are used)
EnableL2TimeCorrection()
EnableL2TSsync()
SetL2TSsyncRate()
EnableLatencyMeasure()

The first 100 IMU packets and first 100 point cloud packets that are received after the connect are skipped.

NOTE: If you are using WSL2 on Windows 11 the easiest setup of network access is to use mirroring. This the easier rather than using virtual networking. In order to do this, you will need to edit or create the. wslconfig file. It is or should be located in the

directory %UserProfile%. It should look something like:
[wsl2]
networkingMode=Mirrored
firewall=false

If you set firewall=true then you will also likely to set some firewall rules to allow the UDP traffic to/from the L2 for the specified ports.
If you edit this while wsl2 is running you will need to use Power Shell to do a wsl –shutdown and restart wsl. Use can use 'ip a' to verify the network interface.

void L2lidar::DisconnectL2()

This closes the UDP socket. The caller can not receive any data from the L2 until a ConnectL2() is performed again.

UDP error reporting

When the connectL2() or any send command methods fails. They only report true or false. If the return is false this method can be used to retrieve the error message associated with that failure.

const QString **GetLastUDPError**()

Retrieve packet methods

These are used to retrieve the latest packet data. These are used by the caller after receiving a signal notification that a new packet was received for that type of packet. These calls are thread safe.

const LidarAckData **ack();**

Get the last ACK packet received.

uint32_t **GetL2Workmode**();

Get the current L2 workmode

const LidarImuDataPacket **imu();**

Get the last IMU packet received.

const LidarIpAddressConfig **IPaddress**()

Get the latest L2 UDP Address config packet received.
Note: This is implemented but has not been observed.

const LidarParamDataPacket **L2ParamsPacket**()

Get the latest L2 parameters packet.

const Lidar2DPointDataPacket **Pcl2Dpacket();**

Get the last 3D point cloud packet received.

const LidarPointDataPacket **Pcl3Dpacket();**

Get the last 3D point cloud packet received.

const LidarTimeStampData **timestamp()**

Get the latest timestamp received.

const LidarVersionData **version();**

Get the last VERSION packet received.

L2 command methods

These are commands sent to the L2.
They are used to:

change the state of the L2
configure L2
reset the L2
request information from the L2 (such as Version info)

bool **GetL2Params**(void)
This sends a request to the L2 for a Parameters packet. When the L2 sends a parameters packet a signal is emitted, L2lidar:: L2ParamsReceived ();

bool **GetWorkMode**();
This generates a request to the L2 to send a parameter packet to obtain the current workmode.

bool L2lidar::**LidarGetVersion**(void)

This sends a request to the L2 for a version packet.
The are 3 posible results:

1. The command is completely ignored (This can happen early in the startup.)
2. An ACK packet is sent with the status: ACK_WAIT_ERROR. This occurs when the L2 is not fully initializes. It will not be fully initialize until the L2 is no longer in standby and the first point cloud packet is sent. If a standby command is sent after the L2 is full initialized then a version packet will still be sent.
3. A VERSION packet is sent (this class will send a signal to any connected subscribers). An ACK packet will also be sent by the L2 with the status ACK_SUCCESS.

bool L2lidar::**LidarReset**(void)

This causes the L2 to restart (should be equivalent to power cycle).
Some 'workmode' changes are only effective after a restart.
Note:
The L2 restart is immediate and does not send and ACK packet that confirms receipt of the command.

bool **LidarStartRotation**(void)

Sends a run command to the L2.

If the L2 was in standby then it should start scanning it can take >20 seconds to come up to speed and start sending point cloud data.

bool **LidarStopRotation**(void)

Send a standby command to the L2.

This causes the L2 to stop the motors and go into low power mode.

Issues have been seen with not being able to bring the L2 out of standby mode without a power cycle.

bool **sendLatencyID**(uint32_t SeqID);

sets packet sequence ID

This is used in measuring the latency of packets over the UDP interface

It sets a sequence ID that is returned in the ACK packet. This allows an ACK packet to be matched to this specific send command.

Note: Command packets and User command packets use the same packet structure

void **LidarSetCmdConfig**(

QString srcIP, uint32_t srcPort,

QString dstIP, uint32_t dstPort);

This command does not communicate with the L2.

It saves the IP setup required to connect to the L2 through UDP.

This will be extended to include the serial com port if UART communications when is implemented.

This must be set before the L2connect() method is used.

bool **SetL2MAC**(LidarMacAddressConfig MACsettings)

This sets the MAC address of the L2. The L2 factory set MAC address that appears not have a Organization Unique Identifier (OUI) that is assigned. It also does not appear to be serialized from L2 unit to L2 unit. You may want to assign a locally managed MAC address. The minimum requirements are:

MAC[0] bit 0x02 must be set (locally managed MAC address)

MAC[0] bit 0x01 must be cleared (L2 is a unicast device)

MAC[1:5] have not restrictions

bool **setL2UDPconfig**(

QString hostIP, uint32_t hostPort,

QString LidarIP, uint32_t LidarPort);

This command sets the UDP configuration used by the L2 to send and receive packets through the Ethernet connection to the L2.

Note:

The L2 does not use DHCP. It is really configured for peer-to-peer operation.

This means the host Ethernet connection should also be manually configured.

It does not appear that the L2 uses jumbo packets.

The factory default when it is shipped are:

Lidar IP: 192.168.1.62

Lidar port: 6101 (this is the port L2 listens for packets)

Host IP: 192.168.1.2

Host port: 6201 (this is the port the L2 uses to send packets)

If you change these parameters, you will also likely need to change the port settings on your host to match.

These settings take effect on the next power cycle or reset of the L2.

bool **SetWorkMode**(uint32_t mode)

This commands only sets the workmode in the L2. It does not restart the L2. This must be done separately for certain workmode changes.

Note:

Immediately effective workmode settings that have been observed are:

Std/Wide FOV

IMU disable/enable

Effective after restart or reset

2D/3D mode

serial/UPD mode

start automatically or wait for start command after power on

NOTE: This uses an undocumented command to perform this function

It was discovered using Wireshark to see how the Unitree software sends commands. This is how the Unitree software sets workmode.

The define LIDAR_PARAM_WORK_MODE_TYPE was added to be consistent with the documented commands.

Packet Stats methods

Packet stats from L2 (only updates when L2 socket connected.)
These are not actually critical, only for reporting stats.

void **ClearCounts**()

This resets the packet count statistics.

uint64_t **lostPackets**()

This returns the total number of lost packets received since the last ClearCounts().

uint64_t **total2D**()

This returns the total number of 2D packets received since the last ClearCounts().

uint64_t **total3D**()

This returns the total number of 3D packets received since the last ClearCounts().

uint64_t **totalACK**()

This returns the total number of ACK packets received since the last ClearCounts().

uint64_t **totalIMU**()

This return the total number of IMU packets received since the last ClearCounts().

uint64_t **totalPackets**()

This returns the total number of packets received since the last ClearCounts().

uint64_t **totalOther**()

This returns the total number of other packets received since the last ClearCounts().

uint64_t **totalIMUretrieved**()

This returns the total number of IMU packets retrieved by the app since the last ClearCounts().

uint64_t **totalPCretrieved**()

This returns the total number of point cloud packets retrieved by the app since the last ClearCounts().

UDP latency measurements

These methods are used to obtain the round trip time latency of UDP packets between the host and the L2.

void **EnableLatencyMeasure**(bool enable)
This enables/disables latency measurements. The disables sending of the L2 latency command. This also means that no ACK packets will be generated.

Latency **GetLatency**()
This gets the most recent UDP RTT latency measurement. The measurements are in msec.

-1.0 is invalid measurement
It return the structure:

```
typedef struct {
   double lastMeasurement; // -1.0 invalid
   double Average; // 0.0 invalid
   double Variance; // 0.0 invalid
   double min; // 999.99 invalid
   double max; // -1.0 invalid
} Latency;
```

The min and max values are the over the entire time period that the L2 is connected using the L2Connect() method.

Time base controls and measurement

The L2 time base does not measure in seconds. It measures in ½ seconds. This is at least true for FW Version 2.8.11.1. The methods are to assist in correcting the L2 time base so time stamps are reported in real world time seconds. These are split into 2 groups.

L2 Time stamp correction and controls

This group is for applying time stamp corrections to incoming IMU, 2D and 3D packets.

void **EnableL2TimeCorrection**(bool enableflag)
if enableflag is false then the time base correction is not applied.
if enableflag is true then the time stamp values returned are scaled.

void **SetL2TimeScale**(long long ScaleNumerator, long long ScaleDenominator)
This set the L2 time base scale. The default are ScaleNumerator=2, ScaleDenominator=1.
If the enable L2 time correction is false or use system timestamp is true then the time units are not scaled. This allows for fine tuning of the time scaling to compensate for the absolute accuracy in the L2 timebase.

void **GetL2TimeScale**(long long& ScaleNumerator, long long& ScaleDenominator)
This returns the current time base scale.

L2 Time base syncing to host

This group methods are used to sync the L2 time base to the host time base.
There are 3 likely scenarios here.

a.) Not used, no attempt to sync to host. In this case these methods are not used
b.) The l2lidar runs a timer and regularly syncs the L2 time base to the host time base. In this case only SetL2TSsyncRate() and EnableL2TSsync() are used.
c.) The calling app will manage syncing the host. In this case only the SyncL2Clock(...) methods are used.

void L2lidar::**EnableL2TSsync**(bool enable)
This method enables a sync timer to operate the a specified rate (default is 20HZ). This will sync the L2 time base to the host time base at the specified rate.

bool L2lidar::**SyncL2Clock**()
This syncs L2 to current host system time.

bool L2lidar::**SyncL2Clock**(TimeStamp timestamp)
This set the L2 time base to the value in the timestamp structure.
The TimeStamp structure is (defined in unitree_lidar.protocol.h):

```
typedef struct {
        uint32_t sec;      // time stamp of second
        uint32_t nsec;     // time stamp of nsecond
} TimeStamp;
```

void L2lidar::**SetL2TSsyncRate**(uint32_t Rate)
This method allows changing the sync rate for the time base from the default. The Rate is in milliseconds. The default rate is 50 milliseconds which gives a 20HZ update rate. Higher rates than this starts to affect the UDP packet latencies.

Override of Range calibration

This group methods are used to override the internal calibration of range values.
The override is enabled/disabled using the method:

```
EnableCalibrationOVR(true); // use the override values for Range Scale/Bias
EnableCalibrationOVR(false);// use the internal calibration values for Range Scale/Bias
```

The values for Range Scale and Range bias are set using:

```
SetCalibration(Scale,Bias);
```

For at least one L2 unit FW: 2.8.11.1:

```
The builtin value Range Scale: 0.000978  // also converts range from mm to meters
The builtin value Range Bias: -365.625    // units: mm
```

Other units with the same FW have reported different calibration data.

If your point cloud data shows barrel distortion, then make Range Bias more negative. If the point cloud data shows pincushion distortion, then make Range Bias more positive.

Readback of override state is done using :

```
bool IsEnabled = GetCalibration(&OverrideScale, &OverrideBias);
```

Signal Methods

These methods are callback functions that are emitted to tell the subscriber that a new packet for that particular type was received. They are notifications only and have no parameters. They use the connect method in the subscriber. The following is an example:

```
connect(&l2lidar, &::L2lidar::PCL3DReceived,
        this, &MainWindow::onNew3DLidarFrame, Qt::QueuedConnection);
```

This example calls the method onNew3DLirdarFrame() when the PCL3DRecieved() notification is emitted from the L2lidar class. When the onNew3DLirdarFrame() is called it would call Pcl3Dpacket() to obtain the latest 3D packet. Similar connects are used for the other signals. These calls are thread safe.
The IMU, 2D and 3D packets notifications occur at the send rate from the L2 for those packets. The send rate for IMU is approximately ~250 HZ. The send rate for the 3D packets is ~220 HZ. The send rate for the 2D packet is ~50 HZ. The send rate for the time stamp received is approx. ~470 HZ. Time stamps are received in every 3D and IMU packet.
A Version packet are only received when LidarGetVersion() is called.
A ACK packet notification is only received when a packet is sent to the L2 with the exception of a 'reset' command.

Note: TimeStamp packets are sent to the L2 but have not been observed received from the L2.

void imuReceived(); IMU packet received

void PCL3DReceived(); 3D point cloud packet received

void PCL2DReceived(); 2D point cloud packet received

void versionReceived(); VERSION packet received

void timestampReceived(); Time stamp update received

void ackReceived(); ACK packet received

void WorkmodeReceived(); L2 workmode updated

void L2ParamsReceived(); L2 parameter packet received

Data Processing Methods

bool **ConvertL2data2pointcloud**(
Frame& frame,
bool Frame3D,
bool IMUadjust,
bool CalOVR,
double CalScale,
double CalBias,
double timeConstraintIMU_PC = 0.07);

This returns the latest point cloud Frame. It obtains the latest raw point cloud packet sent by the L2 and optionally the latest IMU packet. It then converts the raw data from into a point cloud frame. If the IMU adjust is used the it also applies the pose returned by the IMU to the point cloud frame. The point cloud frame will consist of 'n' points. Each point consists s of x,y,z in mm, the time of the point measurement (in epoch time nanoseconds see Note below), the intensity (0-255) and a ring ID (which is always 1 for the L2). The number of points depends on the scan mode, 2D or 3D, and removal of point that are marked invalid. For 3D data that is up to 300 points per frame and for 2D up to 1800 points per frame. It should be called from the parent class when the parent class received a signal that a new frame is available. See the Signal Methods section.

Note: The timestamp for each point is a long long (64 bits). The units are nanoseconds since the Epoch (Thursday 1 January 1970 00:00:00 UT). This was done so that the timestamp does not suffer precision loss. This allows an app to aggregate frames with precise time references for each point. For further processing in environments like ROS2 the app will likely covert the frame or aggregated frame into a separate capture of the start time (first point timestamp) and then make all the point time stamps relative to the start time. Units do not change, they are always nanoseconds

frame is (QVector<PCpoint>)
Frame3D
true process latest 3D packet
false process latest 2D packet
IMUadjust
true applies pose from IMU to point cloud data
false do not applies IMU pose correction

CalOVR

true
Replace the RangeScale and RangeBias calibration values with CalScale and CalBias

false
Use the L2 built-in RangeScale and RangeBias calibration values reported in the point cloud packet.

CalScale

Converts Range value in mm to floating point in meters.
Typical number is 0.000987

CalBias

Offset correction for the Range scaling.
Typical number is -365.625
Adjusting this number has a direct impact on the barrel/pincushion distort you see in your point cloud. You may want to adjust this is there is obvious distortion in your point cloud. The more negative number moves toward pincushion distortion. The more positive the number moves towards barrel distortion.

timeConstraintIMU_PC

Optional parameter, defaults to 0.07 seconds (70 msec).
This parameter is only used if IMUadjust parameter is TRUE.
This is used to decide if the timestamps on the IMU packet and the point cloud packet are close enough to use in the IMU to correct the pose of the point cloud data. This does not change x,y,z translation only the rotation correction to the gravity aligned quaternion in the IMU packet.
Units are in seconds.

return

true successful
false if point cloud packet does not exit
false if IMUAdjust is true and no valid IMU data

The Frame returned has data converted from the azimuth, elevation, range to x,y,z. The returned x,y,z coordinate includes the z offset from the mounting surface to the scan origin.

This means the returned data is referenced to the center of the L2 mounting surface. Users should note that this is not any of the mounting bolt holes but is the center of the circle of the defining the mounting bolt holes. The mounting bolt holes are also offset by 22.5 degrees from the x,y axis origin.

The ’z’ offset is defined by the a_axis_dist L2 calibration parameter.

CLASS PUBLIC VARIABLES

There are no public class variables.

Revision history

V0.3.7

Preliminary release (accidental title version 0.3.8 Release)

V0.3.8 2026-01-29

Release version

Title correction

Added revision history

Moved requestLatencyMeasurement(); from class public to class private.

Removed signal latencyMeasured(double ms);

Added public method GetLatency();

Added Latency structure

Added in class measurement and calculation of RTT latency.

Added timer to send latency commands a regular interval (4 HZ)

V0.3.9 2026-02-01

Changed latest IMU data to be complete IMU packet

Added capability for L2 Time base correction and controls

Added capability for L2 Time base sync to host clock at regular timer interval

Removed Private class section, will replace once documentation for them is stabilized.

V0.3.10 2026-02-02

Added L2 parameters packet support

Added Get workmode

Added enable/disable latency measurements

minor code cleanup

V0.3.11

Added conversion method to turn L2 point data packets into point cloud frames. (This moves the final processing step into the l2lidar class so it doesn't have to be performed by the parent classes.)

Changed get L2 params to use cmd_value = 3 instead of 1. This matches the observed behavior in Unitree's software.

V0.4.1

Clean up of comments and code

Added Set L2 MAC command.

Added Set L2 UDP config.

Changed Get workmode to use a user command to retrive packet as opposed to the undocumented get L2 params command.

Add a single Get workmode command when the app first connects. This was necessary because of Qt's implementation of the readyRead doesn't necesarily accept UDP packets until at least one UD packet is sent.

Added decoders for the 3 config type packets. This is done even though the UDP and MAC packets have not been observed being sent by the L2.

V0.4.2 2026-02-14

Added error messaging reporting

Bug correction in connectL2(), connect flag was being set to late

V0.4.3 2026-02-18

Added more logic to the timestamping of the point cloud data

mL2EnableSyncHost && enableL2TimeStampFix

true use L2 timestamping for each cloud point

false use system time for each cloud point

If adjust cloud packet using IMU is enabled

then if the point cloud packet is different from

IMU packet time by more than 50 msec the point cloud

packet will be dropped.

Added range(m) to point cloud data. It is already present

in the raw point cloud packet. It saves recomputing it later

in a user app. PCpoint.h has been changed to include this field.

Added IMU and PC packet retrieval counts to assist with verifying

packet retrieval by app versus total packets received.

Added more introductory documentation.

V1.0.0 2026-02-20

Separated L2lidar class from the L2diagnostic app and l2lidar_ros2 app

This is the initial release of the standalone L2lidar class

V1.1.0 2026-02-22

Corrected ConvertL2data2pointcloud() to generate more accurate timestamps.

Changed the timestamp units in the cloud point array returned by

ConvertL2data2pointcloud().

PCpoint structure member time change from float to long long

PCpoint structure member time units are now nanoseconds since Epoch

V1.2.0 2026-04-20

Added override of Range Scale and Range calibration params.

V1.3.0 2026-05-12

Changed the timestamp calculations to use long long arithmetic to reduce numerical loss. Allow setting the L2 timebase correction using a/b where a,b are long long types.

V1.3.1 2026-05-17

There are no code changes in this version. This version only affects the documentation. Updates are for:

Networking instructions for wsl2 to use mirroring rather than virtual network.

Clarification of the data returned by the **ConvertL2data2pointcloud()** method.

Minor typo and puncuation.

V1.3.2 2026-05-24

ConvertL2data2pointcloud() now includes optional parameter for time matching constraint between the IMU and point cloud packet when IMUadjust is enabled.